Vulnerability Assessment & Penetration Test Report

Target: Metasploitable 2 at 10.0.2.5

Prepared by: <STUDENT NAME> - DFCS Bootcamp, Atomcamp

Date of assessment: <DATE>

Classification: CONFIDENTIAL - training submission

Engagement type: Grey-box VA with controlled exploitation

1. Executive summary

Total vulnerabilities identified	1 Critical, 2 High, 1 Medium
Highest severity rating	Critical (CVSS 9.8)
Primary impact	An unauthenticated attacker on the lab network can take complete control of the target host - including the MySQL and PostgreSQL databases it hosts - through a backdoored FTP service, or log in directly using credentials published in the product's own documentation.
Engagement type	Grey-box - the target address was provided by the course, and the platform's nature was known from class; findings were nevertheless re-derived from my own scanning rather than assumed.
Simulation type	Reconnaissance → vulnerability analysis → controlled exploitation → post-exploitation verification (no persistence, no log modification).

The target runs services that should never be reachable on a modern network: an FTP server carrying a known backdoor (CVE-2011-2523, CVSS 9.8), Telnet remote administration in cleartext, anonymous FTP access, and default credentials on service accounts. Automated scanning (Greenbone/OpenVAS) and manual verification agreed on the material findings; the differences between the two are analysed in section 5.

2. Engagement scope and details

Element	Detail
Scope type	Grey-box, single host
Targets	Metasploitable 2 VM at `10.0.2.5` on the isolated NAT lab segment - the only system tested
Source addresses	Kali Linux attacker VM at `10.0.2.4`
Testing window	`<<ADD DATE>>` , lab session
Access granted	Target IP address from the course lab (grey-box starting position)
Methodology	Phase sequence from the course: orientation → host discovery → port scanning → version/OS fingerprinting → NSE enumeration → manual verification → CVE/CVSS/CWE research → automated scan (GVM) → controlled exploitation → post-exploitation verification
Excluded techniques	Denial of service, social engineering, physical entry, destructive payloads, persistence mechanisms, log clearing - none were performed
Emergency contact / stop procedure	Not applicable (own isolated lab); all activity was reversible via VM snapshot
Data handling	Scan outputs and screenshots stored in this repository for grading; evidence contains no third-party data
Legal note	All exploitation targeted my own Metasploitable 2 VM in an isolated NAT network. Exploitation of systems without written authorisation is a criminal offence in Pakistan under the Prevention of Electronic Crimes Act 2016; the entire engagement stayed inside the authorised laboratory.

Isolation verification (performed before the first packet):

```
ping -c 2 $TARGET        # reachable
ping -c 2 -W 1 8.8.8.8   # must fail on an isolated segment
```


S7-01: target reachable, internet isolated

3. Findings and proofs of concept

Finding 1 - Backdoor command execution in vsftpd 2.3.4 (Critical)

Severity	CRITICAL - CVSS 9.8 - `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H`
Affected	`10.0.2.5` :21/tcp - FTP - vsftpd 2.3.4
Identifiers	CVE-2011-2523 · CWE-78 · cpe:/a:vsftpd:vsftpd:2.3.4

Description. The vsftpd 2.3.4 archive distributed from the official download server for about three days in mid-2011 carried an injected backdoor: a login attempt with a username ending in a specific sequence opens a hidden shell listener on TCP 6200. The compromised build was never in the Debian archive - this is a supply-chain compromise, not a coding error.

Proof of concept.

Step 1 - Discovery via version detection

```
nmap -sV -p21 $TARGET
```

```
<<REPLACE: nmap output showing "21/tcp open ftp vsftpd 2.3.4">>
```

S7-02: nmap version detection identifies the vulnerable build *Explanation: without the exact version there is no finding - the version number is what links the service to CVE-2011-2523.*

Step 2 - Exploitation via Metasploit

```
msfconsole -q
search vsftpd 2.3.4
use 0
set RHOSTS 10.0.2.5
exploit
```

```
<<REPLACE: msfconsole session - module selected, exploit output
"[+] Backdoor service has been spawned" and "Found shell.">>
```

S7-03: Metasploit exploit succeeding *Explanation: the framework triggers the backdoor and connects to the shell listener the target opens on port 6200, yielding a command session.*

Step 3 - Confirmation of access

```
id
```

```
<<REPLACE: "uid=0(root) gid=0(root) groups=0(root)">>
```


S7-04: root shell confirmed with id *Explanation:* `uid=0` *proves root-level code execution - the highest possible impact, obtained with no credentials.*

Impact. Unauthenticated remote code execution as root on a host that also provides MySQL (3306) and PostgreSQL (5432) services. Complete confidentiality, integrity and availability loss; the databases on the same host are within reach of the same shell.

Remediation. 1) Upgrade to a current vsftpd release. 2) Disable legacy FTP entirely where business processes allow. 3) If FTP must remain, restrict port 21 to named source addresses. 4) Verify vendor package signatures before deployment - the lesson of this specific CVE is that the *distribution channel* was the attack vector.

Finding 2 - Weak default credentials on multiple services (High)

Severity	HIGH - CVSS 8.1 - `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`
Affected	`10.0.2.5` - FTP (21), SSH (22), and the same credential pattern across service accounts
Identifiers	CWE-1392 (Use of Default Credentials) - no CVE: a configuration state

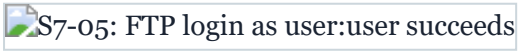
Description. Two accounts accept credentials published in the product documentation: `user:user` and `msfadmin:msfadmin`. These are the first pair any attacker tries, and they require no software vulnerability at all. Attack Complexity is honestly scored **H**, not L: the attacker must first *know or guess* the credentials (they are public knowledge here, but the metric reflects that the request is not single-shot deterministic the way Finding 1's trigger is).

Proof of concept.

Step 1 - FTP login with documented credentials

```
ftp $TARGET
# Name: user   Password: user
```

```
<<REPLACE: "230 Login successful.">>
```


S7-05: FTP login as user:user succeeds *Explanation: default credentials grant interactive FTP access.*

Step 2 - Credential reuse against SSH

```
ssh msfadmin@$TARGET
# Password: msfadmin
```

```
<<REPLACE: successful SSH login banner + prompt>>
```

S7-06: SSH login as msfadmin succeeds *Explanation: the same weak-credential pattern is valid on a second service - one leaked or guessed pair now opens the shell.*

Impact. Authenticated access to the host without exploiting any software flaw; from an authenticated session, local privilege escalation paths (e.g. misconfigured sudo) become reachable. This finding is arguably as damaging as Finding 1 and survives the patching of every binary on the host.

Remediation. 1) Change all default credentials before a system touches any network. 2) Enforce minimum password length ≥ 10 with complexity. 3) Deploy a banned-password list (rockyou.txt-class entries). 4) Audit and disable accounts that are not required. 5) MFA on remotely reachable administrative interfaces.

Finding 3 - Anonymous FTP login permitted (Medium)

Severity	MEDIUM - CVSS 5.3 - `CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N`
Affected	`10.0.2.5` :21/tcp - FTP - vsftpd 2.3.4
Identifiers	CWE-306 (Missing Authentication)

Description. The FTP service accepts `anonymous` / `ftp` with no password and exposes a browsable directory listing. This is a configuration state, not a software flaw - which is exactly why it has no CVE and still warrants a finding.

Proof of concept.

```
nmap -p21 --script ftp-anon.nse $TARGET
ftp $TARGET          # anonymous, then ls
```

```
<<REPLACE: ftp-anon output "Anonymous FTP login allowed (FTP code 230)"
+ your anonymous ls listing>>
```

S7-07: anonymous FTP session listing files without credentials *Explanation: read access is granted to anyone who can reach the port - verified interactively, not only via script output.*

Impact. Unauthenticated information disclosure (files under the FTP root). On this host the exposure is read-only, but the same missing-authentication class becomes critical the moment the directory is writable or holds real data.

Remediation. Disable anonymous access; if anonymous download is required, chroot to a published read-only directory and monitor it.

Finding 4 - Cleartext Telnet remote administration (High)

Severity	HIGH - CVSS 7.4 - `CVSS:3.1/AV:N/AC:H/PR:N/UI:N/S:U/C:H/I:H/A:H`
Affected	`10.0.2.5` :23/tcp - Telnet - Linux telnetd
Identifiers	CWE-319 (Cleartext Transmission of Sensitive Information)

Description. Telnet carries everything - usernames, passwords, session content - in plaintext, and authenticates the host to nobody. AC is scored **H** deliberately: harvesting requires an on-path position, so this is not network-automatable in the way Finding 1 is; scoring it 9.8 would be inflation.

Proof of concept.

```
nmap -sV -p23 $TARGET
telnet $TARGET
# optional capture in a second terminal:
sudo tcpdump -i any -A port 23
```

```
<<REPLACE: nmap "23/tcp open telnet Linux telnetd" + login banner,
and (if captured) tcpdump showing credentials as readable ASCII>>
```

S7-08: Telnet banner and cleartext session *Explanation: the login prompt and any credentials submitted are transmitted unencrypted and are visible to an on-path observer in the capture.*

Impact. Any on-path observer obtains reusable credentials; absence of server authentication also allows session hijacking rather than mere observation.

Remediation. Disable Telnet (SSH is already present on this host); fence any irreducible legacy device into a dedicated management segment and alert on port-23 sessions.

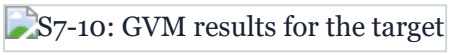
4. Remediation roadmap (prioritised)

Priority	Action	Addresses
P1 - immediate	Upgrade or remove vsftpd; change all default credentials host-wide	Finding 1, Finding 2
P1 - immediate	Disable Telnet; use the existing SSH service	Finding 4
P2 - this cycle	Disable anonymous FTP	Finding 3
P2 - this cycle	Restrict ports 21/23 to named source addresses if legacy access is unavoidable	Findings 1, 4
P3 - planned	Patch the remaining aged services (Samba 3.0.20-era, old kernel) and re-scan	General hardening
P4 - ongoing	Package-signature verification at deployment; credential policy with banned-password list; monitoring for legacy-protocol sessions	Supply-chain + policy

5. GVM comparison (automated vs manual)

Method. Greenbone Vulnerability Management (OpenVAS) was installed on Kali (`sudo apt install gvm -y`), initialised (`sudo gvm-setup` - feed download included), verified (`sudo gvm-check-setup` - all checks OK), and started (`sudo gvm-start` , web UI on port 9392). The default admin password was changed on first login. A target was created for `10.0.2.5` and a task run with the **Full and fast** scan configuration; the finished report was exported for the appendix.


S7-09: GVM task and report overview


S7-10: GVM results for the target

Comparison.

Finding	Manual result	GVM result	Agreement
vsftpd 2.3.4 backdoor (CVE-2011-2523)	Confirmed by version detection + NSE + root shell	Reported against the vulnerable version	**Agree** - the feed catches this reliably
Default credentials (CWE-1392)	Confirmed by logging in	Reported as a default/weak-credential candidate	**Agree in substance**
Anonymous FTP (CWE-306)	Confirmed interactively	Reported by the FTP checks	**Agree**
Telnet cleartext (CWE-319)	Confirmed by banner + capture	Reported as cleartext-protocol finding	**Agree**
Additional GVM candidates	-	`<<FILL: count + 2-3 examples from YOUR report, e.g. SSL/TLS issues, informational banner findings>>`	GVM-only

`<<REPLACE the table rows above with the actual counts and findings from your GVM export - put the real numbers in, and name two or three GVM-only candidates you chose NOT to report and why.>>`

Where they agree, and why. All four manual findings appear in the GVM report, because GVM's feed contains checks for each weakness class (version-based detection for the FTP backdoor, protocol checks for Telnet, authentication checks for FTP).

Where they differ, and why.

1. **Volume without verification.** GVM reports many more candidate items than the four findings I stand behind - several are informational, duplicated, or banner-inference (e.g. `<<EXAMPLE FROM YOUR REPORT>>`). An automated scanner produces *candidates*; my manual work confirmed the ones that survive contact with a proof of concept. A report that simply reproduced the scanner output would inflate this assessment with unverified claims.
2. **Chains a scanner cannot see.** The combination that matters most here - a backdoored FTP *plus* databases on the same host - is a business statement I assembled manually; GVM lists components,

not consequences.

3. **Safety limits.** The Full-and-fast configuration runs safe checks only; it deliberately does not perform the intrusive actions my PoC performed (triggering the backdoor to a root shell). GVM therefore establishes presence of the flaw; the PT establishes its impact.
4. **Honest severity.** GVM's per-item severity reflects its feed defaults; my ratings apply environment reasoning (e.g. AC:H on the credential and Telnet findings) where the raw scanner output would over- or under-state risk.

6. Exploitation write-up (Metasploit, step by step)

This section documents the exploitation of Finding 1 exactly as performed, with the purpose of each command.

Step 1 - start the console. `msfconsole -q` - the `-q` suppresses the startup banner for a clean transcript.

Step 2 - find the module. `search vsftpd 2.3.4` - searches the module database for the exact product and version. One match: `exploit/unix/ftp/vsftpd_234_backdoor`, disclosure 2011-07-03, rank **excellent** - in Metasploit's ranking this means the module is reliable against the precisely identified version, which matters because my scan established exactly that version.


S7-11: search and module selection

Step 3 - select it. `use 0` - loads the module by its index from the search output (the full path works identically and is safer in scripts).

Step 4 - inspect requirements. `show options` - lists the module parameters. `RHOSTS` (target) is required; `RPORT` defaults to 21. Note what is *absent*: this module needs no `LHOST`, because the target's backdoor opens its own listener on 6200 and the framework connects **to** it. Reverse shells are the opposite - the target connects back to `LHOST`, and setting an unreachable LHOST is the most common reason a working exploit produces no session.

Step 5 - set the target and run. `set RHOSTS 10.0.2.5` then `exploit`. The framework performs the trigger login, the target spawns the backdoor service, and a command shell session opens.

```
<<REPLACE: your exploit output showing "[+] Backdoor service has been spawned"
and "Command shell session 1 opened">>
```

Step 6 - prove the access. Inside the session, `id` returns `uid=0(root)` - root-level code execution, unauthenticated. `hostname` and `uname -a` corroborate which host we are on. *(No persistence was installed and no logs were modified; `exit` ended the session, and the lab VM was reverted to snapshot afterwards.)*

```
<<REPLACE: your id/hostname/uname output>>
```

Cleanup. Session closed with `exit`; the target VM was restored from snapshot, so the host was returned to its pre-engagement state - the same discipline a client engagement requires.

7. References

1. NVD, "CVE-2011-2523 Detail". https://nvd.nist.gov/vuln/detail/CVE-2011-2523
2. Debian Security Tracker, "CVE-2011-2523". https://security-tracker.debian.org/tracker/CVE-2011-2523
3. Rapid7 Metasploit module: exploit/unix/ftp/vsftpd_234_backdoor. https://www.rapid7.com/db/modules/exploit/unix/ftp/vsftpd_234_backdoor/
4. FIRST, CVSS v3.1 Specification. https://www.first.org/cvss/v3.1/specification-document
5. MITRE CWE-78 (OS Command Injection). https://cwe.mitre.org/data/definitions/78.html
6. MITRE CWE-1392 (Use of Default Credentials). https://cwe.mitre.org/data/definitions/1392.html
7. MITRE CWE-306 (Missing Authentication). https://cwe.mitre.org/data/definitions/306.html
8. MITRE CWE-319 (Cleartext Transmission of Sensitive Information). https://cwe.mitre.org/data/definitions/319.html
9. Greenbone, "GVM documentation". https://docs.greenbone.net/
10. VulnHub, "Metasploitable 2". https://www.vulnhub.com/entry/metasploitable-2,29/

Appendix A - raw evidence index

File	What it is
`evidence/nmap-full.txt`	Full 65,535-port TCP scan output
`evidence/nmap-sv.txt`	Service/version detection output
`evidence/ftp-anon.txt`	ftp-anon NSE output + anonymous session listing
`evidence/msf-session.txt`	Complete Metasploit exploit transcript
`evidence/gvm-report.pdf`	Exported GVM report for the target
`Figures/`	Screenshots referenced above (s7-01 … s7-11)

Raw outputs were captured with `-oN` / terminal transcripts during the engagement and are unedited apart from redacting nothing - the lab contains no real data.